

# Yukti Software

## Web Application & Platform Architecture Manual

Complete Guide covering React Architecture, Firebase Firestore Integration, Dynamic Data Engine, Multi-Channel Lead Pipeline, SEO Routing & Deployment.

PROJECT NAME

**Yukti Software Official Web Platform**

DATABASE & CLOUD

**Firebase Firestore (v12) NoSQL Engine**

ROUTING STANDARD

**Canonical SEO History API Router (19+ Slugs)**

FRAMEWORK & RUNTIME

**React 18.3 + Vite 5 + Tailwind CSS**

LEAD & AUTOMATION

**EmailJS + Green-API WhatsApp Gateway**

VERSION & STATUS

**v2.0.0 Production Enterprise Edition**

# 1. Executive Overview & Tech Stack

The **Yukti Software Web Platform** is an enterprise Single Page Application (SPA) designed to serve as both an **IT Software Solutions Showcase** and an **Accredited Software Training Institute Portal** based in Greater Noida. The architecture is engineered for ultra-fast load times, robust client-side lead capture, and zero-downtime content delivery through a hybrid database strategy.

| Layer            | Technology                  | Role & Purpose                                                                       |
|------------------|-----------------------------|--------------------------------------------------------------------------------------|
| Frontend Core    | React 18.3.1                | Component-based UI, <code>React.lazy</code> & <code>Suspense</code> code-splitting.  |
| Build Engine     | Vite 5.4.14                 | Sub-second Hot Module Replacement (HMR) and optimized Rollup production builds.      |
| Styling & Theme  | Tailwind CSS 3.4.17         | Utility-first responsive layouts, Glassmorphism, Dark/Light mode theme engine.       |
| Cloud Database   | Firebase Firestore (v12.19) | Real-time NoSQL cloud storage for enquiries, courses, internships, gallery, team.    |
| Email Automation | EmailJS Browser SDK (^4.4)  | Client-side dual email dispatch: Admin notification + User auto-confirmation.        |
| WhatsApp Gateway | Green-API / wa.me link      | Automated instant WhatsApp welcome & syllabus message dispatch to user's phone.      |
| Icons & Micro-UI | Lucide React & Confetti     | SVG icons for all tech stacks + celebration particle animations on form submissions. |

# 2. Directory Structure & Architecture

The repository follows a clean modular organization separating services, data layers, hooks, pages, and components:

```
yukti-main-website/
├─ public/                # Static assets, sitemap.xml, robots.txt, _redirects
├─ src/
│  ├─ assets/             # Vector brand logos, campus photos, team media
│  ├─ components/         # Reusable UI Blocks (Navbar, Hero, Modals, Forms)
│  ├─ context/            # Global ThemeContext (Dark/Light mode)
│  ├─ data/               # Course catalog, syllabus, FAQs, DOCX data
│  ├─ hooks/              # Custom hooks: useInView, useCountUp, useGoogleReviews
│  ├─ pages/              # Lazy-loaded views (AllCourses, Detail, Careers, Gallery)
│  ├─ services/           # Firebase leadService.js and contentService.js
│  ├─ utils/              # Canonical SEO routeUtils.js
│  ├─ App.jsx             # Main application orchestrator & router
│  ├─ data.js             # Global site branding and contact information
│  ├─ firebase.js         # Firebase safe initialization & Firestore export
│  └─ main.jsx            # Application entrypoint
├─ package.json
└─ vite.config.js
```

## 3. Dynamic Data System (How Components Load Data)

---

The web platform implements a **Fail-Safe Hybrid Data Architecture**. Every component that presents dynamic information (courses, careers, campus photos, team members) queries Firebase Firestore first, while keeping a rich local dataset as a real-time fallback.

- 1 Component Mounts (e.g. `AllCoursesPage`, `GalleryPage`, `CareersPage`)
- 2 Invokes `contentService.js` getter function
- 3 Checks if Firebase Firestore is connected & returns live records
- 4 Smart Merge & Deduplication: Merges Firestore items with local catalog
- 5 Instant UI Render: Renders live data seamlessly without blank loading states

### 1. All Courses Page ( `AllCoursesPage.jsx` )

Queries `courses` and `custom_courses` in Firestore. Automatically deduplicates by course slug/title against the 16+ master catalog, providing instant category filtering and real-time search.

### 2. Dynamic Course Detail ( `CourseDetailPage.jsx` )

Dynamically binds route slug (e.g. `course-python`), fetching detailed curriculum modules, duration badges, FAQs, and custom image carousels from `pagesDataFromDocs.js` and `courseImages.js`.

### 3. Careers & Internships ( `CareersPage.jsx` )

Fetches live openings from Firestore `internship_roles`. New roles added via Admin appear instantly with pre-populated application modals.

### 4. Photo & Video Gallery ( `GalleryPage.jsx` )

Merges `gallery_photos` and `gallery_videos` from Firestore with 15+ local workshop assets, categorizing them into Collaborations, Labs, and Events.

## 4. Firebase Integration & Database Architecture

---

### 4.1 Safe Initialization ( `src/firebase.js` )

Firebase is initialized defensively. If environment credentials are missing in `.env`, it gracefully logs a warning and permits the site to function in fallback mode without crashing.

```
import { initializeApp, getApps, getApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID
};

export const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApp();
export const db = getFirestore(app);
```

## 4.2 Firestore Collections Reference

| Collection Name  | Key Document Fields                                                         | Used In                                      |
|------------------|-----------------------------------------------------------------------------|----------------------------------------------|
| enquiries        | name, email, phone, course, service, mode, message, status, serverTimestamp | All Leads, Consultation Modals, Contact Form |
| courses          | key, title, category, duration, badge, color, highlights, modulesCount      | AllCoursesPage, Dropdowns, TrainingCourses   |
| internship_roles | title, tagline, department, badge, keyFocus, technologies, description      | CareersPage, InternshipPage                  |
| gallery_photos   | title, category, caption, image (URL), badge, createdAt                     | GalleryPage Photo Tab & Modal                |
| gallery_videos   | title, student, role, duration, videoUrl, thumbnail, badge                  | GalleryPage Video Testimonials               |
| team_members     | name, role, education, experience, image, specialties, stats                | Team Component & About Page                  |

## 5. Unified Multi-Channel Lead Engine ( `leadService.js` )

When any student or enterprise client fills a form on the website, `leadService.submitEnquiry()` triggers a synchronized 4-tier dispatch pipeline:

Synchronized Lead Dispatch Steps:

1. Database Record:

Creates document in Firestore `enquiries` collection with `serverTimestamp()` .

2. Admin Email Notification:

Dispatches real-time email alert to admissions team via EmailJS.

3. User Auto-Reply:

Sends branded confirmation auto-reply email to candidate's mailbox.

4. WhatsApp Automated Message:

Green-API pushes instant WhatsApp greeting with batch details to user's mobile number.

5. Interactive Feedback:

Triggers `canvas-confetti` and renders success confirmation modal.

## 6. SEO & Canonical SPA Routing Engine

To guarantee high search engine rankings for competitive Greater Noida keywords, the application maps every page and master course to its exact canonical SEO slug in `src/utils/routeUtils.js` .

| Page / Course Topic          | Canonical URL Slug                                         | Internal Key                       |
|------------------------------|------------------------------------------------------------|------------------------------------|
| Home / Landing               | <code>/</code>                                             | <code>home</code>                  |
| About Us                     | <code>/about-us/</code>                                    | <code>about</code>                 |
| Careers & Internships        | <code>/carrer-page/</code>                                 | <code>careers</code>               |
| All Courses Master Directory | <code>/courses/</code>                                     | <code>courses</code>               |
| Software Training Institute  | <code>/software-training-institute-greater-noida/</code>   | <code>training-institute</code>    |
| Python Training Course       | <code>/python-training-institute-greater-noida/</code>     | <code>course-python</code>         |
| Java Full Stack Course       | <code>/java-full-stack-course-greater-noida/</code>        | <code>course-java-fullstack</code> |
| MERN Stack Course            | <code>/mern-stack-training-institute-greater-noida/</code> | <code>course-mern-stack</code>     |

| Page / Course Topic        | Canonical URL Slug                                | Internal Key |
|----------------------------|---------------------------------------------------|--------------|
| DSA & System Design        | /dsa-course-greater-noida/                        | course-dsa   |
| AI & Machine Learning      | /ai-and-machine-learning-course-greater-noida/    | course-ai-ml |
| Database Management (DBMS) | /database-management-system-course-greater-noida/ | course-dbms  |

**Navigation Mechanics:** Handled seamlessly using `window.history.pushState` and `popstate` listeners. Users can click browser Back/Forward or refresh any deep URL without breaking state or requiring server-side rendering.

## 7. How to Start, Run, Build & Deploy

---

### 7.1 Environment Setup ( `.env` )

```
# Firebase Configuration
VITE_FIREBASE_API_KEY=your_api_key_here
VITE_FIREBASE_AUTH_DOMAIN=your_project.firebaseio.com
VITE_FIREBASE_PROJECT_ID=your_project_id
VITE_FIREBASE_STORAGE_BUCKET=your_project.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=your_sender_id
VITE_FIREBASE_APP_ID=your_app_id

# EmailJS Configuration
VITE_EMAILJS_SERVICE_ID=your_service_id
VITE_EMAILJS_PUBLIC_KEY=your_public_key
VITE_EMAILJS_ADMIN_TEMPLATE_ID=your_admin_template_id
VITE_EMAILJS_USER_TEMPLATE_ID=your_user_template_id

# Green-API WhatsApp Gateway (Optional)
VITE_GREEN_API_ID_INSTANCE=your_instance_id
VITE_GREEN_API_TOKEN_INSTANCE=your_token_instance
```

### 7.2 CLI Commands

```
# 1. Install Dependencies
npm install

# 2. Start Local Development Server (http://localhost:5173)
npm run dev

# 3. Create Optimized Production Build
npm run build

# 4. Preview Production Build Locally
npm run preview
```

### 7.3 Deployment Guidelines

#### Netlify / Vercel / Cloudflare

Pre-configured with `public/_redirects` containing: `/* /index.html 200`. Deploy repository directly with build command `npm run build` and publish directory `dist`.

#### Firebase Hosting / Apache cPanel

For Firebase, run `firebase init hosting` and set public directory to `dist`. For Apache, verify `.htaccess` includes standard rewrite rules to `index.html`.

### Production Verification Checklist:

- Run `npm run build` to confirm 0 compilation errors.
- Ensure all environment variables in hosting provider match `.env` file.
- Submit a test enquiry from Consultation Modal and verify Firestore record and email delivery.

---

**Yukti Software — Tech Innovation Hub, Greater Noida, NCR, India**

Helpline: +91 95828 15419 | Email: [contact@yuktisoftware.com](mailto:contact@yuktisoftware.com) | Web: <https://yuktisoftware.com>